1. Proposed Architecture

Identify the primary architecture pattern your application uses or will use.
Examples include:

- Layered architecture
- Client-server architecture
- Model-View-Controller (MVC)
- Microservices
- Event-driven architecture

Include:

- The name of the architecture pattern
- A brief explanation of why it fits your application
- A diagram showing the major layers, components, or services
- The direction of communication among components

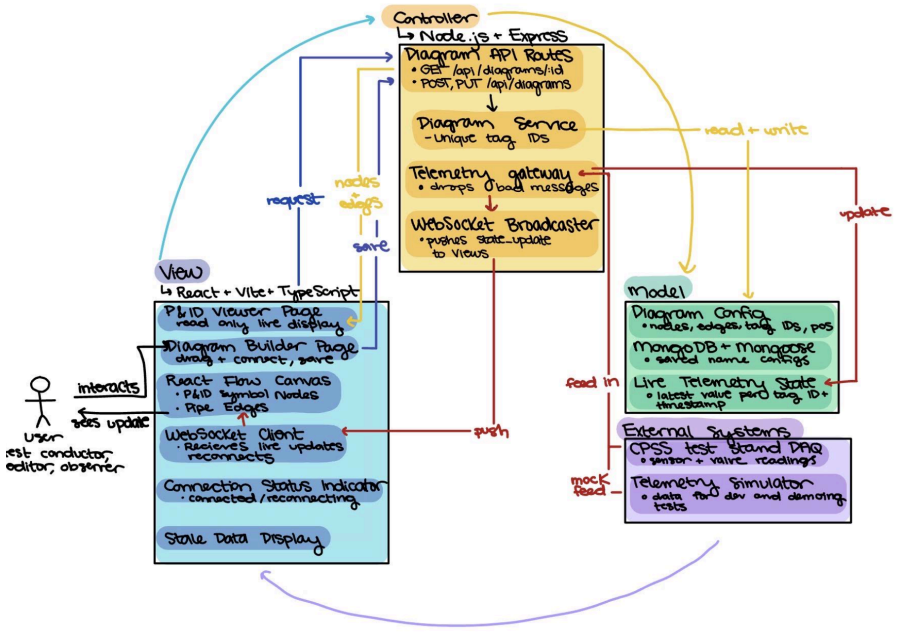
For example, a MERN application might show:
React client -> Express API -> service layer -> MongoDB database

Your diagram should also show important external systems, such as Supabase Auth, Stripe, Google Maps, or a notification service, if your app uses them.

1. Proposed Architecture

Model-View-Controller (MVC)

The MVC architecture pattern fits DE-Nile because users interact with a visual P&ID dashboard editor while the system separately manages saved diagram data and live telemetry. The React interface of the P&ID editor is the **View**, the RiverDash side receives the live data and validates the data which is the **Controller**, and the database that stores the P&ID layouts, telemetry, and data from RiverDash is the **Model**.



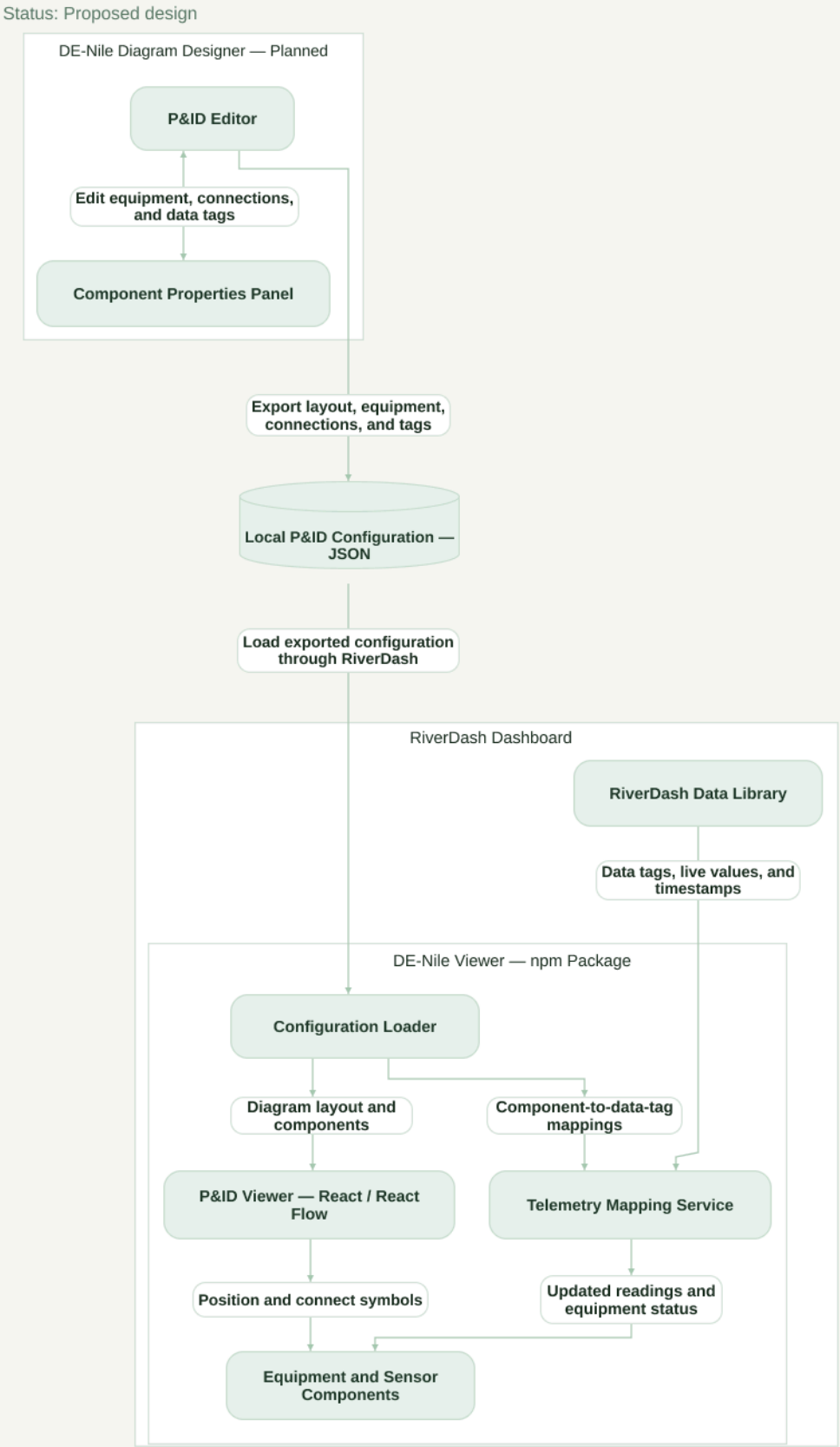
2. High-Level Design Diagram

Create a high-level design diagram that shows the major parts of your application.
Include:

- Main client-side components or pages
- Backend routes, controllers, or services
- Data storage
- Important external APIs or systems
- Key data flows between components

Do not show every function or database field. Focus on the major parts of the system and how they work together.

Status: Proposed design



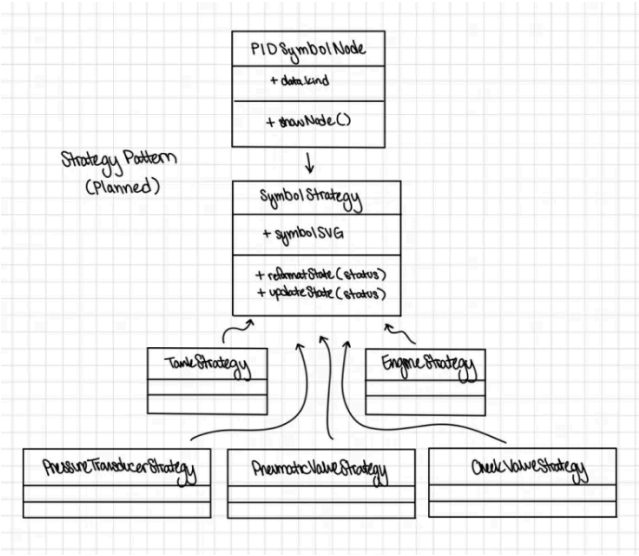
3. Design Patterns

Planned Pattern: Strategy
We plan to implement the Strategy Design Pattern because it'll help us handle the different ways P&ID symbols will be displayed and updated depending on the status and data from the test stand.

We need to display different symbols for 5 component types: tanks, check valves, pneumatic valves, pressure transducers, and rocket engines. Each type needs different SVG files, labels, and changes states depending on its status (i.e. tanks will appear red if low on fuel). However, the problem with this is that one large PIDSymbolNode component would be full of long if/else condition statements which is difficult to test, read, and maintain. Thus, if we implement the Strategy Design Pattern then we can break down each type of component into its own strategy containing the rules of its own equipment type.

Current Components:
App.tsx - Responsible for creating the React Flow Nodes. Each node contains a data.kind (i.e "tank" or "check-valve")
PIDSymbolNode.tsx - The main component containing everything related to Nodes such as the data.kind, SVG files, position, connections, colors, etc.
Planned Component:
SymbolStrategies - A separate component that stores the rules for each symbol type and anything related to that node such as it's SVG images, states, colors, etc.

Small UML Diagram:



4. Design Decisions and Tradeoffs

Briefly describe at least two design decisions your team made.
For each decision, explain:

- What options your team considered
- Which option your team selected
- Why that choice fits the project's current needs

Examples:

- Using Supabase Auth rather than building authentication from scratch
- Using a service layer to separate business logic from Express routes
- Using an Adapter to work with a third-party maps API
- Using the Observer pattern to send delivery-status notifications

Decision 1: Using React Flow with custom SVG Nodes

Our team was considering building our own HTML editor with PNG Nodes or using React Flow with SVG Nodes. We decided to select the React Flow with custom SVG Nodes because the React Flow import supports dragging, connecting, moving, zooming in and out, and had a lot of pre-existing functions that supported the visual editor we're trying to make which would save us a lot of time from trying to build the editor ourselves with HTML. We also decided to choose SVG over PNG because SVG is encoded vectors that stay high quality even upon zooming in and out so it won't have quality issues like with PNG.

Decision 2: Only Monitoring & Editing Diagram

Our team was considering whether to allow the user to send status updates and control physical components of the test stand such as the valves or keep it limited to only monitoring and editing the P&ID diagram of the test stand. We decided to select the Only Monitoring & Editing Diagram option because it's safer, lowers the project's complexity, and prevents a bug in the diagram display from causing problems with the physical test stand.